Algorithm Report

Quick Sort

Algorithm Choice: Chosen because it is an efficient comparison-based sorting algorithm with excellent average-case performance. It is suitable for large datasets and is commonly used in practice.

Time Complexity: Average: O(n log n), Worst: $O(n^2)$, Best: O(n log n)

Space Complexity: Average: O(log n) recursion, Worst: O(n)

Binary Search

Algorithm Choice: Chosen because it searches a sorted array very efficiently by repeatedly dividing the search space in half.

Time Complexity: Best: O(1), Average/Worst: O(log n)

Space Complexity: O(1)

0/1 Knapsack (Dynamic Programming)

Algorithm Choice: Chosen because dynamic programming guarantees the optimal solution by storing solutions to overlapping subproblems.

Time Complexity: O(n × W), where n = number of items and W = knapsack capacity

Space Complexity: O(n × W)